

İstanbul Nişantaşı Üniversitesi

EBLG362 - Derin Öğrenime Giriş

Tensorflow ile Churn Modeli Geliştirme Projesi

Bilgisayar Mühendisliği

Cenker Sipahi | 20222013213 - Gizem Başköy | 20222013228



Müşteri Kaybı (Churn) Tahmin Projesi



1. Proje Amacı ve İçeriği

Bu çalışma, bir telekomünikasyon şirketinin müşteri verilerini analiz ederek, hangi müşterilerin hizmet almayı bırakacağını (**Churn**) önceden tahmin etmek amacıyla geliştirilmiştir.

Veri Kaynağı: Bu projede kullanılan veri seti [Kaggle - Telco Customer Churn \(IBM\)](#) üzerinden temin edilmiştir.

Proje Kapsamı:

- Veri Analizi:** Müşteri demografisi ve kullanım alışkanlıklarının incelenmesi.
- Veri Ön İşleme:** Eksik verilerin temizlenmesi ve kategorik verilerin sayısallaştırılması.
- Model Eğitimi:** **TensorFlow/Keras** ile derin öğrenme tabanlı bir Yapay Sinir Ağları (ANN) kurulması.
- Optimizasyon:** Yapılan analizler sonucunda modelin ezberlemesini (overfitting) önlemek adına eğitimin **25 epoch** ile sınırlandırılması.
- Tahminleme:** Mevcut verilerle gelecekteki müşteri davranışlarının öngörülmesi.



2. Kullanılan Teknolojiler

- Veri İşleme:** Pandas , Numpy
- Makine Öğrenmesi:** Scikit-Learn (Scaling & Splitting)
- Derin Öğrenme:** TensorFlow & Keras (Yapay Sinir Ağları)
- Görselleştirme:** Matplotlib , Seaborn

Verileri tablo şeklinde görüntülemek ve üzerinde düzenlemeler yapmak için **pandas** kullanıyoruz.

Veriler üzerinde matematiksel işlemleri yapmak için **numpy** kullanılır.

1. Veri Seti Hazırlığı ve Temizliği

Bu aşamada verideki eksik değerleri kontrol ediyor ve model için anlam taşımayan **ID** gibi sütunları temizliyoruz.

Not: Sayısal olması gereken **TotalCharges** sütunundaki hatalı formatlar bu bölümde düzeltilmiştir.

```
import pandas as pd
import numpy as np

# Veri setini okuyoruz
# '../data/' ifadesi: bir üst klasöre çık ve data klasörüne gir demektir
df = pd.read_csv('../data/WA_Fn-UseC_-Telco-Customer-Churn.csv')

# İlk 5 satırı kontrol edelim
df.head()
```

Bu kısımda **Kaggle** üzerinden aldığımız veri setinin içeriğini kontrol ediyoruz.

[1]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	...	No	No	No	No	Month-to-month	Yes
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	...	Yes	No	No	No	One year	No
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes	...	No	No	No	No	Month-to-month	Yes
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	...	Yes	Yes	No	No	One year	No
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	...	No	No	No	No	Month-to-month	

5 rows × 21 columns

Veri setimizin içeriği görselde görüldüğü şekildedir, ancak bu veri şu anda sadece bizim için anlamlı. Bilgisayarın da anlaması için ileri adımlarda bu verileri sayısal değere çevireceğiz.

```
# Veri tiplerini ve eksik değerleri kontrol et
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column                Non-Null Count  Dtype
---  --
 0   customerID            7043 non-null   str
 1   gender                7043 non-null   str
 2   SeniorCitizen         7043 non-null   int64
 3   Partner               7043 non-null   str
 4   Dependents            7043 non-null   str
 5   tenure                7043 non-null   int64
 6   PhoneService          7043 non-null   str
 7   MultipleLines         7043 non-null   str
 8   InternetService       7043 non-null   str
 9   OnlineSecurity        7043 non-null   str
10   OnlineBackup          7043 non-null   str
11   DeviceProtection      7043 non-null   str
12   TechSupport           7043 non-null   str
13   StreamingTV           7043 non-null   str
14   StreamingMovies       7043 non-null   str
15   Contract              7043 non-null   str
16   PaperlessBilling      7043 non-null   str
17   PaymentMethod         7043 non-null   str
18   MonthlyCharges        7043 non-null   float64
19   TotalCharges          7043 non-null   str
20   Churn                 7043 non-null   str
dtypes: float64(1), int64(2), str(18)
memory usage: 1.1 MB
```

```
# customerID sütununu siliyoruz
df.drop('customerID', axis=1, inplace=True)
print("Sütun silindi. Yeni sütun sayısı:", len(df.columns))
```

Sütun silindi. Yeni sütun sayısı: 20

Bu kısımda yine **pandas** sayesinde verilerin türlerini görebiliyoruz. Görüldüğü üzere 21 adet veri tipimiz var. Bunlardan 18 tanesi **String**, 2 tanesi **Integer** ve 1 tanesi de **Float** veri tipindedir.

Bu veri setinde customerID verisine ihtiyacımız olmadığı için o sütünü **pandas** ile siliyoruz.

```
[4]: # Boşluk içeren satırları bulup onları NaN (boş) yapıyoruz, sonra sütunu sayıya çeviriyoruz
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')

# Boş kalan satırları (NaN) temizliyoruz
df.dropna(inplace=True)

print("Veri sayıya çevrildi ve boş satırlar temizlendi.")
```

Veri sayıya çevrildi ve boş satırlar temizlendi.

2. Verilerin Sayısallaştırılması

Bilgisayarın metinleri işleyebilmesi için **One-Hot Encoding** yöntemi kullanılarak kategorik değişkenler sayısal verilere dönüştürülmüştür.

```
[5]: # 'Churn' (ayrılma durumu) sütununu 1 ve 0 yapalım
df['Churn'] = df['Churn'].apply(lambda x: 1 if x == 'Yes' else 0)

# Diğer tüm metin sütunlarını otomatik olarak sayısal verilere (kukla değişkenlere) çevirelim
df_final = pd.get_dummies(df)

# Son halini kontrol et
df_final.head()
```

```
[5]:
```

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges	Churn	gender_Female	gender_Male	Partner_No	Partner_Yes	Dependents_No	...	StreamingMovies_Yes	Contract_Month-to-month	Contract_One year	Contract_Two year	PaperlessBillin
0	0	1	29.85	29.85	0	True	False	False	True	True	...	False	True	False	False	
1	0	34	56.95	1889.50	0	False	True	True	False	True	...	False	False	True	False	
2	0	2	53.85	108.15	1	False	True	True	False	True	...	False	True	False	False	
3	0	45	42.30	1840.75	0	False	True	True	False	True	...	False	False	True	False	
4	0	2	70.70	151.65	1	True	False	True	False	True	...	False	True	False	False	

5 rows × 46 columns

Bu kısımda da görüldüğü üzere ve önceden bahsettiğimiz gibi verileri sayısallaştırıyoruz. Görüldüğü üzere Int, Float ve Boolean olmak üzere verileri değiştirdik ve String tipinden kurtulduk.

Boşluk içeren yani hiçbir veri bulunmayan satırları da tespit edip NaN olarak atayıp boş olduğunu belirtiyoruz.

3. Veriyi Bölme ve Ölçeklendirme (Min-Max Scaling)

Bu aşama, modelin kararlı çalışması için iki kritik işlem içerir:

- **Eğitim ve Test Ayırımı:** Veri setini %80 oranında eğitim, %20 oranında test olacak şekilde bölüyoruz.
- **Veri Ölçeklendirme (Scaling):** Verilerimizdeki farklı sayısal aralıkları (örneğin; aylık ücretler ile kullanım süresi) 0 ile 1 arasına sıkıştırıyoruz.
 - **Önemli:** `fit_transform` fonksiyonu ile eğitim setindeki kuralı öğreniyor, `transform` ile bu kuralı test setine uyguluyoruz.

Not: Bu işlem yapılmazsa, büyük sayısal değerler model üzerinde haksız bir ağırlık oluşturarak tahminleri bozabilir.

```
[6]: from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split

# 1. Veriyi Giriş (X) ve Hedef (y) olarak ayırılım
X = df_final.drop('Churn', axis=1)
y = df_final['Churn']

# 2. Veriyi Eğitim ve Test olarak ayırılım (%80 - %20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Ölçeklendirme
scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Veri %80 Eğitim ve %20 Test olarak ayarlandı!")
```

Veri %80 Eğitim ve %20 Test olarak ayarlandı!

Verinin hepsini kullanmayacağımız için bu verinin %80'ini eğitim aşamasında, %20'sini ise asıl performansını görmek için test aşamasına saklıyoruz. Model bu %20'lik dilimi eğitim boyunca görmeyecektir.



4. Yapay Sinir Ağı (ANN) Mimarisi

Modelimiz, verideki gizli desenleri öğrenmek için ardışık katmanlardan oluşur:

1. **Giriş Katmanı:** 45 parametre girişi.
2. **Gizli Katmanlar:** Karmaşıklığı çözmek için 20 ve 15 nöronlu **ReLU** katmanları.
3. **Çıkış Katmanı:** **Sigmoid** aktivasyonu ile 0-1 arası olasılık tahmini.

```
[7]: import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# 1. Modelin Mimarisini Kuruyoruz
from tensorflow.keras.layers import Input # Input katmanını içeri aktar

model = Sequential([
    Input(shape=(45,)),          # Önce giriş boyutunu ayrı bir katman olarak tanımla
    Dense(15, activation='relu'), # Sonra nöronları ekle
    Dense(5, activation='relu'),
    Dense(1, activation='sigmoid')
])

# 2. Modeli Derliyoruz
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

# 3. Eğitimi Başlatıyoruz (50 Epoch)
history = model.fit(X_train, y_train, epochs=50, validation_split=0.2)
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuD
lugin.
Epoch 1/50
141/141 ————— 1s 2ms/step - accuracy: 0.6544 - loss: 0.7018 - val_accuracy: 0.7671 - val_loss: 0.6604
Epoch 2/50
141/141 ————— 0s 2ms/step - accuracy: 0.7749 - loss: 0.6442 - val_accuracy: 0.7929 - val_loss: 0.6253
Epoch 3/50
141/141 ————— 0s 1ms/step - accuracy: 0.7933 - loss: 0.6132 - val_accuracy: 0.7973 - val_loss: 0.5991
Epoch 4/50
141/141 ————— 0s 1ms/step - accuracy: 0.7953 - loss: 0.5877 - val_accuracy: 0.8036 - val_loss: 0.5721
Epoch 5/50
141/141 ————— 0s 2ms/step - accuracy: 0.8000 - loss: 0.5653 - val_accuracy: 0.8071 - val_loss: 0.5510
Epoch 6/50
141/141 ————— 0s 2ms/step - accuracy: 0.8002 - loss: 0.5461 - val_accuracy: 0.8116 - val_loss: 0.5334
```

Bu aşamada modeli oluşturuyoruz ve eğitiyoruz.
Ne kadar katman olacağı, hangi aktivasyon fonksiyonunun kullanılacağını, katman sayısını bu kısımda tanımlıyoruz.

Buna ek olarak modeli derlerken gözetlemek için kullanacağımız metrik ve optimizier gibi değerleri, ne kadar epoch yapılacağını bu kısımda belirliyoruz.

```
[8]: # 1. Test verileriyle modeli değerlendirelim
test_loss, test_acc = model.evaluate(X_test, y_test)
print(f"Test Doğruluğu: %{test_acc*100:.2f}")

# 2. İlk 5 test örneği için tahmin yapalım
predictions = model.predict(X_test)

# Tahminleri (olasılıkları) 1 ve 0'a çevirelim
y_pred = [1 if p > 0.5 else 0 for p in predictions]

print("İlk 5 Tahmin:", y_pred[:5])
print("Gerçek Değerler:", y_test[:5].values)

44/44 ————— 0s 1ms/step - accuracy: 0.7882 - loss: 0.4458
Test Doğruluğu: %78.82
44/44 ————— 0s 1ms/step
İlk 5 Tahmin: [0, 0, 1, 0, 0]
Gerçek Değerler: [0 0 1 0 0]
```

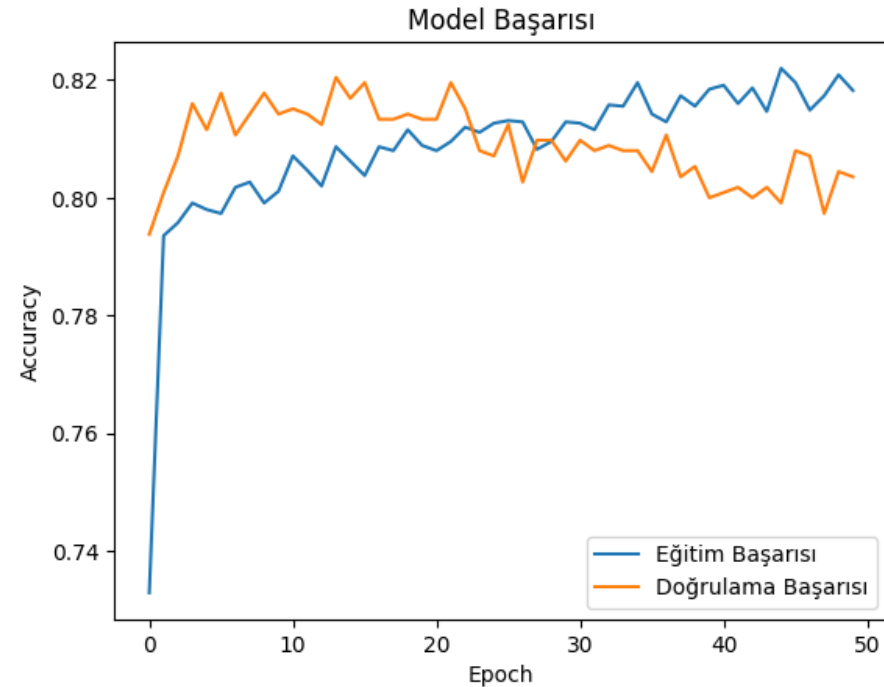
Eğitimden sonra ayırdığımız %20'lik test verisini kullanarak modelin asıl doğruluğunu sınıyoruz. Burada model gördüğü veriyle ilk kez karşılaştığı için doğruluk oranını olabildiğince yükseltmeyi hedefliyoruz.

5. Model Eğitim Performansı

Aşağıdaki grafikler, modelin her adımda (**Epoch**) ne kadar geliştiğini, hata oranının nasıl düştüğünü göstermektedir.

```
[19]: import matplotlib.pyplot as plt
```

```
# Doğruluk (Accuracy) grafiği  
plt.plot(history.history['accuracy'], label='Eğitim Başarısı')  
plt.plot(history.history['val_accuracy'], label='Doğrulama Başarısı')  
plt.title('Model Başarısı')  
plt.xlabel('Epoch')  
plt.ylabel('Accuracy')  
plt.legend()  
plt.show()
```



Eğitim ve Testi birbiriyle karşılaştırmayı kolaylaştırmak için **matplotlib.pyplot** ile elde ettiğimiz sonuçları karşılaştırabiliriz.

Grafikte görüldüğü üzere **20-25 Epoch** civarı bizim için ideal çünkü ondan sonrası **overfitting'e** doğru yol alıyor. **Bu durumu düzeltmek için Epoch** sayısı düşürülebilir, modelin mimarisi (**Katman Sayısı, Aktivasyon Fonksiyonu**) veya **Veri Seti** değiştirilebilir/gözden geçirilebilir.

Model Değerlendirmesi: En Optimal Nokta Analizi

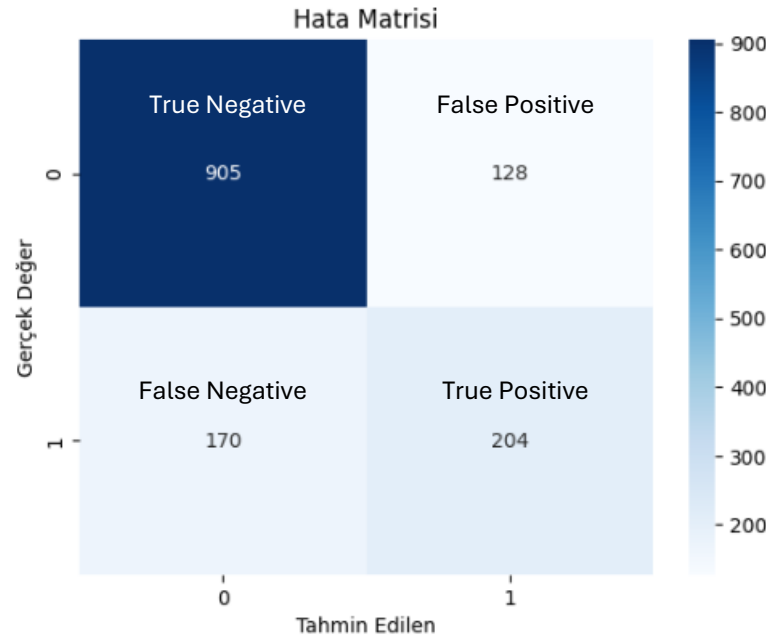
Yukarıdaki başarı ve hata (Accuracy & Loss) grafiklerini incelediğimizde, modelin performansı için **25. epoch** civarının en optimal nokta (Sweet Spot) olduğu gözlemlenmiştir.

Neden 25. Epoch?

- **Denge:** Bu noktada Eğitim (Train) ve Doğrulama (Validation) başarıları birbirine en yakın seviyededir.
- **Genelleme Yeteneği:** 25. epoch'tan sonra mavi çizgi yükselmeye devam etse de turuncu çizginin sabitlendiği görülmektedir. Bu, modelin bu noktadan sonra veriyi öğrenmek yerine yavaş yavaş "ezberlemeye" (**Overfitting**) başladığının bir işaretidir.
- **Karar:** Modelin gerçek dünya verilerinde (hiç görmediği müşterilerde) en yüksek isabetle çalışması için 25-30 epoch arası bir eğitimin bu veri seti için en sağlıklı sonuç olduğu sonucuna varılmıştır.

```
[10]: from sklearn.metrics import confusion_matrix
import seaborn as sns

cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Tahmin Edilen')
plt.ylabel('Gerçek Değer')
plt.title('Hata Matrisi')
plt.show()
```



Aynı şekilde sonuçları hata matrisi ile de gözetleyebiliriz.

- **Sol Üst (905 - True Negative):** Modelin "**ayrılmayacak**" dediği ve **gerçekten ayrılmayan** sadık müşteriler. Bu kısım en yüksek sayıya sahip, yani modelin sadık müşterileri tanımada oldukça başarılı.
- **Sağ Alt (204 - True Positive):** Modelin "**ayrılacak**" dediği ve **gerçekten aboneliğini iptal eden** müşteriler. Bunlar doğru tahmin edilen kayıplardır.
- **Sağ Üst (128 - False Positive):** Modelin "**ayrılacak**" diye yanlış alarm verdiği ama aslında **sistemde kalan** müşteriler.
- **Sol Alt (170 - False Negative):** Modelin "**kalacak**" dediği ama aslında bizi bırakıp **giden müşteriler**.

▼ 6. Modelin Dışa Aktarılması

Eğitilen model, ileride gerçek zamanlı tahminler yapmak üzere `churn_modeli.keras` adıyla kaydedilmiştir.

```
[11]: model.save('../models/churn_modeli.keras')  
      print("Model en güncel formatta kaydedildi!")
```

Model en güncel formatta kaydedildi!

7. Model ve Ölçeklendirme Anahtarının Kaydedilmesi

Projenin en önemli adımı burasıdır. Sadece modeli değil, veriyi modele uygun hale getiren **Scaler** nesnesini de kaydediyoruz:

- **Model Kaydı:** Eğitilen yapay zeka beynini `.keras` formatında saklıyoruz.
- **Scaler Kaydı:** Yeni gelecek verileri, modelin anladığı 0-1 arasına dönüştürmek için kullandığımız "ölçeklendirme anahtarını" `joblib` ile kaydediyoruz.

Neden Gerekli? Eğer bu dosyaları kaydetmezsek, Notebook'u kapattığımızda modelin tüm öğrenmesi silinir ve yeni bir müşteri için tahmin yapamayız.

```
[12]: import joblib  
  
      # Scaler nesnesini (ölçeklendirme anahtarını) kaydediyoruz  
      joblib.dump(scaler, 'scaler.pkl')  
  
      print("Ölçeklendirme anahtarı (scaler.pkl) başarıyla kaydedildi!")
```

Ölçeklendirme anahtarı (scaler.pkl) başarıyla kaydedildi!

Son olarak modeli `.keras` olarak kaydedip kullanabiliriz.

EBLG362 - Derin Öğrenime Giriş

Dinlediğiniz İçin Teşekkürler.

Projenin GitHub Linki

https://github.com/gizembsky/Churn_Analizi_Projesi

